

JAVA PROGRAMMING – COMPLETE DETAILED EXAM SOLUTIONS

SECTION – I

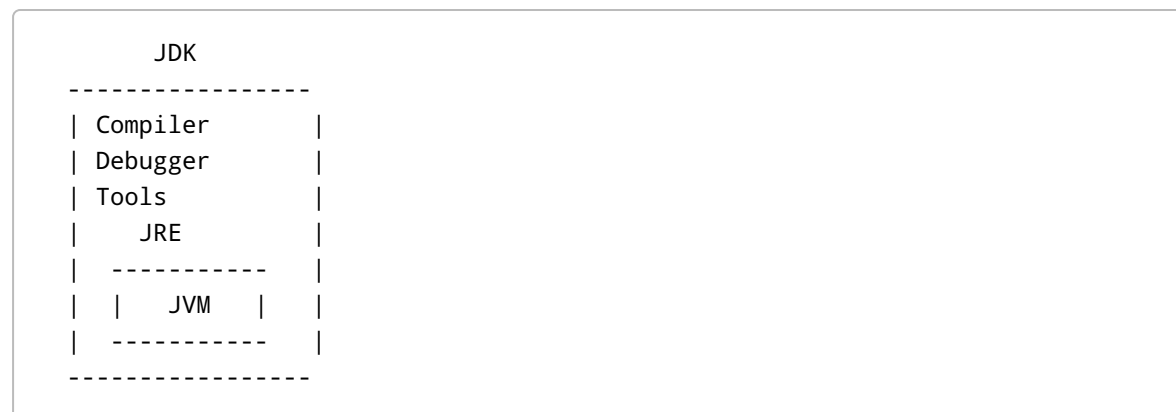
Q1 (a) What is the relation between JVM, JRE and JDK? (2 Marks)

Definition

Java platform consists of three important components: 1. JVM (Java Virtual Machine) 2. JRE (Java Runtime Environment) 3. JDK (Java Development Kit)

These work together to compile and run Java programs.

Block Diagram



JVM (Java Virtual Machine)

Definition

JVM is a virtual machine that executes Java bytecode.

Key Features

- Platform dependent
- Executes .class files
- Provides memory management

- Provides security

Functions

- Loads class files
 - Verifies bytecode
 - Executes code
 - Garbage collection
-

JRE (Java Runtime Environment)

Definition

JRE provides the environment required to run Java applications.

Components

- JVM
- Java class libraries
- Supporting files

Use

Used only for running Java applications.

JDK (Java Development Kit)

Definition

JDK is a complete development package for Java programming.

Components

- JRE
- Compiler (javac)
- Debugger
- Documentation tools

Use

Used for developing and running Java applications.

Relationship

JDK = JRE + Development Tools
JRE = JVM + Libraries

Real Life Example

- JVM = Engine
 - JRE = Car environment
 - JDK = Complete car factory with tools
-

Q1 (b) Differentiate between static and dynamic binding. (2 Marks)

Definition

Binding means connecting a method call to method body.

Difference Table

Static Binding	Dynamic Binding
Done at compile time	Done at runtime
Faster	Slower
Used in method overloading	Used in method overriding
Private, static and final methods	Overridden methods
Early binding	Late binding

Example of Static Binding

```
class Test {  
    static void display() {  
        System.out.println("Static Binding");  
    }  
  
    public static void main(String args[]) {  
        display();  
    }  
}
```

Output

Static Binding

Example of Dynamic Binding

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Main {  
    public static void main(String args[]) {  
        Animal a = new Dog();  
        a.sound();  
    }  
}
```

Output

Dog barks

Q1 (c) Show usage of 'super' keyword with suitable example. (2 Marks)

Definition

super keyword refers to the immediate parent class object.

Uses of super keyword

1. Access parent class variable
 2. Call parent class method
 3. Call parent class constructor
-

Example Program

```
class Animal {
    String color = "White";

    Animal() {
        System.out.println("Animal Constructor");
    }

    void eat() {
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    String color = "Black";

    Dog() {
        super();
        System.out.println("Dog Constructor");
    }

    void display() {
        System.out.println("Child Color: " + color);
        System.out.println("Parent Color: " + super.color);

        super.eat();
    }
}

class Main {
    public static void main(String args[]) {
        Dog d = new Dog();
        d.display();
    }
}
```

Expected Output

```
Animal Constructor  
Dog Constructor  
Child Color: Black  
Parent Color: White  
Animal eats food
```

Q1 (d) Explain the role of finally keyword. (2 Marks)

Definition

finally block is used to execute important code whether exception occurs or not.

Characteristics

- Executes always
- Used for cleanup activities
- Runs after try-catch
- Commonly used for closing files and database connections

Syntax

```
try {  
    // risky code  
}  
catch(Exception e) {  
    // handling code  
}  
finally {  
    // always executed  
}
```

Example Program

```
class Test {  
    public static void main(String args[]) {  
        try {  
            int a = 10 / 0;  
        }  
        catch(ArithmeticException e) {  
            System.out.println("Exception handled");  
        }  
        finally {  
            System.out.println("Finally block executed");  
        }  
    }  
}
```

Output

```
Exception handled  
Finally block executed
```

Advantages

- Resource management
- Prevents memory leakage
- Ensures code execution

Q1 (e) Differentiate between Swing and AWT frameworks. (2 Marks)

AWT	Swing
Heavyweight components	Lightweight components
Platform dependent	Platform independent
Less rich UI	Rich UI components
Uses native OS controls	Written completely in Java
Faster	Slightly slower

AWT

Swing

Limited components

Advanced components

Example Components

AWT

- Button
- Label
- TextField

Swing

- JButton
- JLabel
- JTextField
- JTable
- JTree

Real Life Application

Swing is used in:

- Desktop applications
- IDEs
- Banking software

Q1 (f) How list in Java is different from normal array? Explain with example. (2 Marks)

Difference Table

Array	List
Fixed size	Dynamic size
Stores similar data	Stores objects
Faster	Flexible
Cannot add/remove dynamically	Can add/remove dynamically

Example Program

```
import java.util.*;

class Test {
    public static void main(String args[]) {

        // Array
        int arr[] = {10,20,30};

        // List
        ArrayList<Integer> list = new ArrayList<Integer>();

        list.add(10);
        list.add(20);
        list.add(30);
        list.add(40);

        System.out.println("Array Length: " + arr.length);
        System.out.println("List Size: " + list.size());
    }
}
```

Output

```
Array Length: 3
List Size: 4
```

Q1 (g) Explain javap tool with an example. (2 Marks)

Definition

javap is a Java disassembler tool used to display bytecode information of class files.

Key Features

- Displays bytecode
 - Shows methods and variables
 - Helps in debugging
 - Used for understanding JVM instructions
-

Syntax

```
javap ClassName
```

Example

Java Program

```
class Demo {  
    int x = 10;  
  
    void display() {  
        System.out.println(x);  
    }  
}
```

Command

```
javac Demo.java  
javap Demo
```

Output

```
Compiled from "Demo.java"  
class Demo {  
    int x;  
    Demo();
```

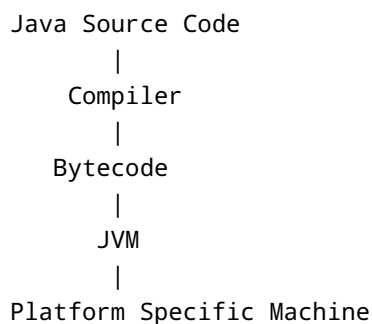
```
void display();  
}
```

Q2 (a) What are the key mechanisms employed by JVM to ensure both platform independence and security? (7 Marks)

Definition

JVM provides platform independence and security using bytecode execution, verification, class loading and sandboxing.

Block Diagram



Mechanisms for Platform Independence

1. Bytecode

- Java compiler converts source code into bytecode.
- Bytecode can run on any JVM.

Advantage

"Write Once Run Anywhere"

2. JVM

- JVM interprets bytecode.

- Different operating systems have different JVMs.
-

3. JIT Compiler

- Converts bytecode into machine code.
 - Improves performance.
-

Mechanisms for Security

1. Class Loader

Loads classes dynamically.

Types

- Bootstrap loader
 - Extension loader
 - Application loader
-

2. Bytecode Verifier

Checks:

- Illegal code
 - Invalid memory access
 - Stack overflow issues
-

3. Security Manager

Controls access to:

- Files
 - Network
 - Memory
-

4. Sandbox Environment

Restricts untrusted programs.

Advantages

- Secure execution
 - Platform independence
 - Better portability
 - Reduced system crashes
-

Real Life Applications

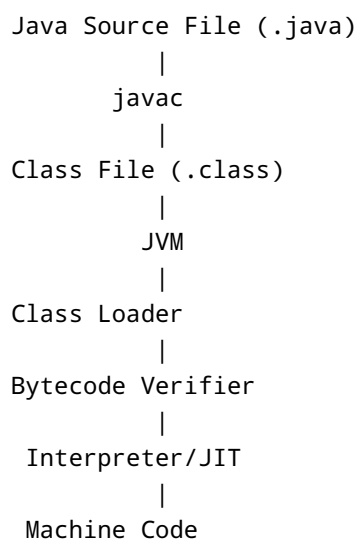
- Android applications
 - Banking systems
 - Enterprise applications
-

Q2 (b) Illustrate compilation and execution process of Java program with flow chart. (7 Marks)

Definition

Java execution process converts source code into bytecode and then machine code.

Flow Chart



|
Output

Stepwise Explanation

Step 1 – Write Program

Program is written in .java file.

Example:

```
class Hello {  
    public static void main(String args[]) {  
        System.out.println("Hello Java");  
    }  
}
```

Step 2 – Compilation

Command:

```
javac Hello.java
```

Compiler creates:

```
Hello.class
```

Step 3 – Class Loading

JVM loads bytecode into memory.

Step 4 – Verification

Bytecode verifier checks security.

Step 5 – Execution

Interpreter/JIT executes code.

Output

Hello Java

Advantages

- Platform independent
 - Secure execution
 - Easy portability
-

Q3 (a) Discuss security architecture of JVM. (7 Marks)

Definition

JVM security architecture protects Java applications from malicious code and unauthorized access.

Security Architecture Diagram

```
graph TD; A[Class Loader] --> B[Bytecode Verifier]; B --> C[Security Manager]; C --> D[Access Controller]; D --> E[Secure Execution];
```

Components

1. Class Loader

Loads class files securely.

Characteristics

- Dynamic loading
 - Namespace separation
 - Prevents unauthorized classes
-

2. Bytecode Verifier

Checks:

- Type safety
 - Illegal conversion
 - Stack access
-

3. Security Manager

Controls sensitive operations.

Examples:

- File access
 - Network access
 - Memory access
-

4. Access Controller

Checks permissions.

Advantages

- Prevents virus attacks
 - Prevents memory corruption
 - Safe internet applications
-

Limitations

- Additional overhead
 - Slightly slower execution
-

Real Life Applications

- Online banking
 - Secure web applications
 - Enterprise servers
-

Q3 (b) Discuss various data types available in Java. Explain size of each data type. What type casting options are available among them? (14 Marks)

Definition

Data types specify the type of data that variables can store.

Types of Data Types

1. Primitive Data Types

Data Type	Size	Range
byte	1 byte	-128 to 127
short	2 bytes	-32768 to 32767
int	4 bytes	Large integer
long	8 bytes	Very large integer
float	4 bytes	Decimal values
double	8 bytes	Large decimal values
char	2 bytes	Unicode character
boolean	1 bit	true/false

2. Non Primitive Data Types

- String
 - Array
 - Class
 - Interface
-

Example Program

```
class DataTypes {  
    public static void main(String args[]) {  
  
        byte b = 10;  
        short s = 200;  
        int i = 5000;  
        long l = 100000L;  
  
        float f = 12.5f;  
        double d = 55.555;  
  
        char c = 'A';  
        boolean flag = true;  
  
        System.out.println(b);  
        System.out.println(s);  
        System.out.println(i);  
        System.out.println(l);  
        System.out.println(f);  
        System.out.println(d);  
        System.out.println(c);  
        System.out.println(flag);  
    }  
}
```

Output

```
10  
200  
5000  
100000  
12.5  
55.555
```

```
A  
true
```

Type Casting

Definition

Converting one data type into another.

Types of Casting

1. Widening Casting

Automatic conversion.

```
byte → short → int → long → float → double
```

Example

```
int a = 10;  
double d = a;
```

2. Narrowing Casting

Manual conversion.

```
double d = 55.5;  
int x = (int)d;
```

Type Casting Diagram

```
byte → short → int → long → float → double
```

Advantages

- Better memory management
 - Flexible programming
-

Limitations

- Data loss in narrowing
 - Precision loss
-

SECTION – II

Q4 Create a Java program to calculate area of shapes using method overloading. (14 Marks)

Definition

Method overloading means using same method name with different parameters.

Key Features

- Compile time polymorphism
 - Improves readability
 - Reusability
-

Formulae Used

Shape	Formula
Circle	πr^2
Rectangle	$l \times b$
Triangle	$0.5 \times b \times h$

Block Diagram

```
User Input
  |
ShapeCalculator Class
  |
Overloaded area() Methods
  |
Display Result
```

Java Program

```
import java.util.Scanner;

class ShapeCalculator {

    // Area of Circle
    double area(double radius) {
        return 3.14 * radius * radius;
    }

    // Area of Rectangle
    int area(int length, int breadth) {
        return length * breadth;
    }

    // Area of Triangle
    double area(float base, float height) {
        return 0.5 * base * height;
    }
}

class Main {
    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);
        ShapeCalculator obj = new ShapeCalculator();

        try {
            // Circle
            System.out.println("Enter radius:");
            double r = sc.nextDouble();

            if(r < 0)
                throw new ArithmeticException("Invalid Radius");
        }
    }
}
```

```

        System.out.println("Area of Circle = " + obj.area(r));

        // Rectangle
        System.out.println("Enter length and breadth:");
        int l = sc.nextInt();
        int b = sc.nextInt();

        System.out.println("Area of Rectangle = " + obj.area(l,b));

        // Triangle
        System.out.println("Enter base and height:");
        float base = sc.nextFloat();
        float h = sc.nextFloat();

        System.out.println("Area of Triangle = " + obj.area(base,h));
    }

    catch(Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}
}

```

Concept Used in Code

Method Overloading

Same method name area() with different parameters.

Exception Handling

Used to handle invalid inputs.

Scanner Class

Used for user input.

Expected Output

```

Enter radius:
5
Area of Circle = 78.5

Enter length and breadth:

```

```
4 5
Area of Rectangle = 20

Enter base and height:
10 6
Area of Triangle = 30.0
```

Advantages

- Code reusability
- Easy maintenance
- Better readability

Real Life Applications

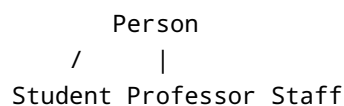
- Calculator software
- Geometry systems
- Engineering applications

Q5 Develop a Java program to simulate a university system using inheritance. (14 Marks)

Definition

Inheritance allows one class to acquire properties of another class.

Class Hierarchy Diagram



Key Features

- Reusability
- Method overriding

- Hierarchical inheritance
-

Java Program

```
// Parent Class
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

// Student Class
class Student extends Person {
    int studentID;

    Student(String name, int age, int studentID) {
        super(name, age);
        this.studentID = studentID;
    }

    void enrollCourse() {
        System.out.println(name + " enrolled in Java course");
    }

    void displayInfo() {
        super.displayInfo();
        System.out.println("Student ID: " + studentID);
    }
}

// Professor Class
class Professor extends Person {
    String department;

    Professor(String name, int age, String department) {
        super(name, age);
        this.department = department;
    }
}
```



```

    void gradeAssignment() {
        System.out.println(name + " graded assignments");
    }

    void displayInfo() {
        super.displayInfo();
        System.out.println("Department: " + department);
    }
}

// Staff Class
class Staff extends Person {
    String role;

    Staff(String name, int age, String role) {
        super(name, age);
        this.role = role;
    }

    void manageOffice() {
        System.out.println(name + " manages office work");
    }
}

class University {
    public static void main(String args[]) {

        Student s = new Student("Aman",20,101);
        Professor p = new Professor("Ravi",45,"CSE");
        Staff st = new Staff("Sohan",35,"Clerk");

        s.displayInfo();
        s.enrollCourse();

        System.out.println();

        p.displayInfo();
        p.gradeAssignment();

        System.out.println();

        st.displayInfo();
        st.manageOffice();
    }
}

```

Concepts Used

Inheritance

Child classes inherit Person class.

Method Overriding

DisplayInfo method overridden.

super Keyword

Used to call parent constructor.

Expected Output

```
Name: Aman  
Age: 20  
Student ID: 101  
Aman enrolled in Java course
```

```
Name: Ravi  
Age: 45  
Department: CSE  
Ravi graded assignments
```

```
Name: Sohan  
Age: 35  
Sohan manages office work
```

Advantages

- Reduces duplication
 - Improves maintenance
 - Better organization
-

SECTION – III

Q6 (a) Compare creating threads by extending Thread class versus implementing Runnable interface in Java. (7 Marks)

Definition

Thread is a lightweight subprocess used for multitasking.

Method 1 – Extending Thread Class

Example

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
  
    public static void main(String args[]) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

Output

```
Thread is running
```

Method 2 – Implementing Runnable Interface

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Runnable thread running");  
    }  
}
```

```
public static void main(String args[]) {  
    MyRunnable obj = new MyRunnable();  
    Thread t = new Thread(obj);  
    t.start();  
}  
}
```

Output

```
Runnable thread running
```

Comparison Table

Thread Class	Runnable Interface
Inheritance used	Interface used
Cannot extend another class	Can extend another class
Less flexible	More flexible
Tight coupling	Better design
Less reusable	Highly reusable

Advantages of Runnable

- Better resource sharing
 - Multiple inheritance support
 - Recommended approach
-

Real Life Applications

- Gaming
 - Banking systems
 - Multitasking applications
-

Q6 (b) Discuss importance of layout managers in Java. Compare BorderLayout, FlowLayout, GridLayout and CardLayout. (7 Marks)

Definition

Layout managers arrange GUI components automatically.

Importance

- Automatic arrangement
 - Responsive design
 - Platform independence
 - Better UI management
-

Layout Manager Diagram

```
Container
|
Layout Manager
|
Components Arrangement
```

1. FlowLayout

Characteristics

- Left to right arrangement
- Default layout of Panel

Example

```
setLayout(new FlowLayout());
```

Real Life Use

Simple forms.

2. BorderLayout

Characteristics

- Five regions
- North
- South
- East
- West
- Center

Diagram

```
NORTH
WEST CENTER EAST
SOUTH
```

Example

```
setLayout(new BorderLayout());
```

3. GridLayout

Characteristics

- Rows and columns
- Equal size components

Example

```
setLayout(new GridLayout(2,2));
```

4. CardLayout

Characteristics

- Multiple screens
- One component visible at a time

Real Life Application

ATM interface.

Comparison Table

Layout	Arrangement	Use
FlowLayout	Left to right	Forms
BorderLayout	5 regions	Main windows
GridLayout	Grid structure	Calculator
CardLayout	Multiple pages	Wizard forms

Q7 (a) Evaluate necessity of organizing classes into packages in Java. Discuss access modifiers. (7 Marks)

Definition

Package is a collection of related classes and interfaces.

Advantages of Packages

- Better organization
 - Avoids naming conflict
 - Improves security
 - Reusability
 - Easy maintenance
-

Package Syntax

```
package mypack;
```

Access Modifiers

Modifier	Same Class	Same Package	Subclass	Other Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Example Program

```
package mypack;

public class Test {
    public int a = 10;
    private int b = 20;

    public void display() {
        System.out.println(a);
        System.out.println(b);
    }
}
```

Advantages

- Encapsulation
- Better security
- Controlled access

Real Life Applications

- Enterprise applications
- Banking software

Q7 (b) Develop a Java program using custom exceptions. (7 Marks)

Definition

Custom exceptions are user-defined exceptions.

Key Features

- Better error handling
 - Application specific messages
 - Improves debugging
-

Java Program

```
import java.util.Scanner;

// Custom Exception
class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) {
        super(msg);
    }
}

class Test {
    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        try {
            System.out.println("Enter age:");
            int age = sc.nextInt();

            if(age < 18)
                throw new InvalidAgeException("Age must be 18 or above");

            System.out.println("Eligible for voting");
        }

        catch(InvalidAgeException e) {
            System.out.println("Custom Exception: " + e.getMessage());
        }
    }
}
```

```
        catch(Exception e) {  
            System.out.println("Invalid Input");  
        }  
    }  
}
```

Output

```
Enter age:  
15  
Custom Exception: Age must be 18 or above
```

Concepts Used

- Exception handling
- throw keyword
- Custom exception class

SECTION – IV

Q8 (a) Write short notes on List Iterator with example programs. (7 Marks)

Definition

ListIterator is used to traverse list elements in both forward and backward directions.

Key Features

- Bidirectional traversal
 - Add/remove elements
 - Modify elements
-

Methods

Method	Description
hasNext()	Checks next element
next()	Returns next element
hasPrevious()	Checks previous element
previous()	Returns previous element

Example Program

```
import java.util.*;

class Test {
    public static void main(String args[]) {

        ArrayList<String> list = new ArrayList<String>();

        list.add("Java");
        list.add("Python");
        list.add("C++");

        ListIterator<String> itr = list.listIterator();

        System.out.println("Forward Traversal");

        while(itr.hasNext()) {
            System.out.println(itr.next());
        }

        System.out.println("Backward Traversal");

        while(itr.hasPrevious()) {
            System.out.println(itr.previous());
        }
    }
}
```

Output

```
Forward Traversal
Java
```

```
Python
C++
Backward Traversal
C++
Python
Java
```

Advantages

- Easy traversal
- Dynamic modification
- Better collection handling

Q8 (b) Write short notes on HashMap with example programs. (7 Marks)

Definition

HashMap stores data in key-value pairs.

Characteristics

- Fast retrieval
- Uses hashing
- No duplicate keys
- Allows null values

Syntax

```
HashMap<Key, Value> map = new HashMap<>();
```

Example Program

```
import java.util.*;

class Test {
```

```
public static void main(String args[]) {  
  
    HashMap<Integer,String> map = new HashMap<Integer,String>();  
  
    map.put(1,"Aman");  
    map.put(2,"Rahul");  
    map.put(3,"Ravi");  
  
    System.out.println(map);  
  
    System.out.println("Value at key 2: " + map.get(2));  
}  
}
```

Output

```
{1=Aman, 2=Rahul, 3=Ravi}  
Value at key 2: Rahul
```

Advantages

- Fast searching
- Efficient storage
- Easy key-value mapping

Real Life Applications

- Database indexing
- Caching systems
- Dictionary implementation

Q9 (a) Discuss I/O streams in Java. Elaborate on stream filters and buffered streams. (7 Marks)

Definition

I/O streams are used for input and output operations.

Types of Streams

1. Byte Streams

- InputStream
- OutputStream

2. Character Streams

- Reader
- Writer

I/O Stream Diagram

Input Device → Input Stream → Program → Output Stream → Output Device

Filter Streams

Definition

Filter streams process data before reading/writing.

Examples

- DataInputStream
- DataOutputStream
- PrintStream

Buffered Streams

Definition

Buffered streams improve performance using buffer memory.

Examples

- BufferedReader
- BufferedWriter
- BufferedInputStream

- BufferedOutputStream
-

Example Program

```
import java.io.*;

class Test {
    public static void main(String args[]) throws Exception {

        FileWriter fw = new FileWriter("test.txt");
        BufferedWriter bw = new BufferedWriter(fw);

        bw.write("Hello Java");

        bw.close();

        FileReader fr = new FileReader("test.txt");
        BufferedReader br = new BufferedReader(fr);

        System.out.println(br.readLine());

        br.close();
    }
}
```

Output

```
Hello Java
```

Advantages

- Faster file handling
 - Efficient memory usage
 - Reduced disk access
-

Q9 (b) Illustrate how PL/SQL stored procedures are created with an example. (7 Marks)

Definition

Stored procedure is a named PL/SQL block stored in database.

Characteristics

- Reusable
 - Faster execution
 - Better security
-

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name
AS
BEGIN
    -- statements
END;
```

Example

Table

```
CREATE TABLE Employee(
    empid NUMBER,
    name VARCHAR2(20),
    salary NUMBER
);
```

Stored Procedure

```
CREATE OR REPLACE PROCEDURE showEmployee
AS
BEGIN
```

```
FOR r IN (SELECT * FROM Employee)
LOOP
    DBMS_OUTPUT.PUT_LINE(r.empid || ' ' || r.name || ' ' || r.salary);
END LOOP;
END;
/
```

Execute Procedure

```
BEGIN
    showEmployee;
END;
/
```

Expected Output

```
101 Aman 50000
102 Ravi 60000
```

Advantages

- Reduces network traffic
 - Better modularity
 - Secure database operations
-

Real Life Applications

- Banking systems
 - Payroll systems
 - Student management systems
-

IMPORTANT KEYWORDS SUMMARY

Definitions Covered

✓ JVM ✓ JRE ✓ JDK ✓ Binding ✓ Inheritance ✓ Threads ✓ Layout Managers ✓ Packages ✓ HashMap ✓ ListIterator ✓ I/O Streams ✓ Stored Procedures

Concepts Covered

✓ Method Overloading ✓ Method Overriding ✓ Exception Handling ✓ Custom Exceptions ✓ Threads ✓ Collection Framework ✓ Swing and AWT ✓ GUI Layouts ✓ Java Security ✓ Type Casting

ADDITIONAL JAVA EXAM PAPER SOLUTIONS

Q1 (a) Discuss portability in the context of Java. (2 Marks)

Definition

Portability means Java programs can run on different platforms without modifying the source code.

Key Features

- Write Once Run Anywhere (WORA)
 - Platform independent
 - Uses bytecode
 - JVM support
-

Block Diagram

```
Java Source Code
  |
  v
Compiler
  |
  v
Bytecode
  |
  v
JVM on Windows/Linux/Mac
```

Explanation

Java compiler converts source code into bytecode. Bytecode runs on any operating system having JVM.

Advantages

- Easy software deployment
- Reduced development cost
- Cross-platform compatibility

Real Life Applications

- Android apps
- Banking software
- Enterprise systems

Q1 (b) Differentiate between method overloading and method overriding. (2 Marks)

Method Overloading	Method Overriding
Same method name with different parameters	Same method name and parameters
Compile-time polymorphism	Runtime polymorphism
Occurs in same class	Requires inheritance
Faster execution	Slightly slower
Return type may differ	Return type must be same

Example of Method Overloading

```
class Test {  
  
    void add(int a, int b) {
```

```
        System.out.println(a+b);
    }

    void add(int a, int b, int c) {
        System.out.println(a+b+c);
    }
}
```

Example of Method Overriding

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
```

Q1 (c) Write a small code snippet to show constructor overloading. (2 Marks)

Definition

Constructor overloading means multiple constructors with different parameters.

Java Program

```
class Student {

    Student() {
        System.out.println("Default Constructor");
    }

    Student(String name) {
        System.out.println("Student Name: " + name);
    }
}
```

```
public static void main(String args[]) {  
    Student s1 = new Student();  
    Student s2 = new Student("Aman");  
}  
}
```

Output

```
Default Constructor  
Student Name: Aman
```

Q1 (d) What are two different ways of creating threads in Java? (2 Marks)

Definition

Thread is a lightweight subprocess.

Methods of Creating Threads

1. Extending Thread Class

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread Running");  
    }  
}
```

2. Implementing Runnable Interface

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Runnable Running");  
    }  
}
```

Advantages

- Multitasking
 - Better CPU utilization
 - Concurrent execution
-

Q1 (e) What is MVC? Explain in brief. (2 Marks)

Definition

MVC stands for:

- Model
- View
- Controller

It is a software design pattern.

MVC Diagram

```
User
|
View ↔ Controller ↔ Model
```

Components

Model

Handles data and business logic.

View

Handles user interface.

Controller

Controls communication between Model and View.

Advantages

- Better code organization
 - Easy maintenance
 - Reusability
-

Real Life Applications

- Spring MVC
 - Web applications
 - Desktop applications
-

Q1 (f) Discuss types of JDBC drivers. (2 Marks)

Definition

JDBC drivers connect Java applications with databases.

Types of JDBC Drivers

Type	Driver Name	Characteristics
Type 1	JDBC-ODBC Bridge	Slow, obsolete
Type 2	Native API Driver	Uses native libraries
Type 3	Network Protocol Driver	Middleware based
Type 4	Thin Driver	Pure Java, fastest

Diagram

```
graph TD
    A[Java Application] --> B[JDBC Driver]
    B --> C[Database]
```

Most Common Driver

Type 4 Driver.

Q1 (g) Differentiate between Enumerator and Iterator. (2 Marks)

Enumerator	Iterator
Legacy interface	Modern interface
Used with Vector	Used with collections
Read-only access	Read and remove elements
Less flexible	More flexible
Methods: hasMoreElements(), nextElement()	Methods: hasNext(), next()

Example of Iterator

```
import java.util.*;

class Test {
    public static void main(String args[]) {
        ArrayList<String> list = new ArrayList<String>();

        list.add("Java");
        list.add("Python");

        Iterator<String> itr = list.iterator();

        while(itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}
```


Output

Java
Python

Q2 (a) Analyze security challenges associated with client-side programming in Java. (7 Marks)

Definition

Client-side Java programming faces various security threats in web applications.

Common Security Threats

1. Cross Site Scripting (XSS)

Definition

Attacker injects malicious scripts into web pages.

Prevention

- Input validation
- Output encoding
- Secure frameworks

2. Cross Site Request Forgery (CSRF)

Definition

Unauthorized requests performed using authenticated users.

Prevention

- CSRF tokens
- Secure sessions
- SameSite cookies

3. SQL Injection

Definition

Malicious SQL queries inserted into forms.

Prevention

- PreparedStatement
 - Input sanitization
-

Example using PreparedStatement

```
import java.sql.*;

class Test {
    public static void main(String args[]) throws Exception {

        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/test","root","root");

        String query = "select * from user where id=?";

        PreparedStatement ps = con.prepareStatement(query);

        ps.setInt(1,101);

        ResultSet rs = ps.executeQuery();

        while(rs.next()) {
            System.out.println(rs.getString(1));
        }
    }
}
```

Security Features of Java

- Bytecode verification
 - JVM sandbox
 - Security manager
 - Automatic memory management
-

Advantages

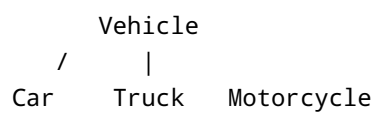
- Secure execution
- Controlled resource access
- Safe network communication

Q4 Design a Java program to model hierarchy of vehicles using inheritance. (14 Marks)

Definition

Inheritance allows one class to acquire properties of another.

Class Hierarchy Diagram



Java Program

```
// Parent Class
class Vehicle {

    private String make;
    private String model;
    private int year;

    Vehicle(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }

    public String getMake() {
        return make;
    }

    public String getModel() {
        return model;
    }
}
```

```

    }

    public int getYear() {
        return year;
    }

    void display() {
        System.out.println(make + " " + model + " " + year);
    }
}

// Car Class
class Car extends Vehicle {

    int numDoors;

    Car(String make, String model, int year, int numDoors) {
        super(make, model, year);
        this.numDoors = numDoors;
    }

    void display() {
        super.display();
        System.out.println("Doors: " + numDoors);
    }
}

// Truck Class
class Truck extends Vehicle {

    double payloadCapacity;

    Truck(String make, String model, int year, double payloadCapacity) {
        super(make, model, year);
        this.payloadCapacity = payloadCapacity;
    }

    void display() {
        super.display();
        System.out.println("Payload Capacity: " + payloadCapacity);
    }
}

// Motorcycle Class
class Motorcycle extends Vehicle {

    Motorcycle(String make, String model, int year) {
        super(make, model, year);
    }
}

```

```
class Main {  
    public static void main(String args[]) {  
  
        Car c = new Car("Hyundai", "i20", 2022, 4);  
        Truck t = new Truck("Tata", "TruckX", 2020, 5000);  
        Motorcycle m = new Motorcycle("Honda", "Shine", 2021);  
  
        c.display();  
        t.display();  
        m.display();  
    }  
}
```

Concepts Used

Inheritance

Child classes inherit Vehicle.

Encapsulation

Private variables with getters.

Polymorphism

display() method overridden.

Expected Output

```
Hyundai i20 2022  
Doors: 4  
  
Tata TruckX 2020  
Payload Capacity: 5000.0  
  
Honda Shine 2021
```

Advantages

- Code reuse
- Better organization

- Easy maintenance
-

Q5 Develop a calculator using method overloading. (14 Marks)

Definition

Method overloading means same method name with different parameters.

Supported Operations

- Addition
 - Subtraction
 - Multiplication
 - Division
-

Block Diagram

```
graph TD
    A[User Input] --> B[Calculator Class]
    B --> C[Overloaded Methods]
    C --> D[Result]
```

Java Program

```
import java.util.Scanner;

class Calculator {

    // Addition
    int add(int a, int b) {
        return a+b;
    }

    double add(double a, double b) {
```

```

        return a+b;
    }

    // Subtraction
    int sub(int a, int b) {
        return a-b;
    }

    // Multiplication
    int mul(int a, int b) {
        return a*b;
    }

    // Division
    double div(int a, int b) {

        if(b == 0)
            throw new ArithmeticException("Division by zero not allowed");

        return a/b;
    }
}

class Main {
    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);
        Calculator c = new Calculator();

        try {
            System.out.println("Enter two integers:");

            int a = sc.nextInt();
            int b = sc.nextInt();

            System.out.println("Addition = " + c.add(a,b));
            System.out.println("Subtraction = " + c.sub(a,b));
            System.out.println("Multiplication = " + c.mul(a,b));
            System.out.println("Division = " + c.div(a,b));
        }

        catch(Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

Expected Output

```
Enter two integers:  
10 5  
Addition = 15  
Subtraction = 5  
Multiplication = 50  
Division = 2.0
```

Advantages

- Better readability
- Code reusability
- Compile-time polymorphism

Q6 (a) Explain importance of thread synchronization in Java. (7 Marks)

Definition

Synchronization controls access of multiple threads to shared resources.

Need of Synchronization

- Prevents race condition
- Maintains data consistency
- Avoids thread interference

Race Condition Diagram

```
Thread 1 -----> Shared Resource <----- Thread 2
```


Java Program

```
class Counter {  
  
    int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
}  
  
class Test {  
    public static void main(String args[]) throws Exception {  
  
        Counter c = new Counter();  
  
        Thread t1 = new Thread(() -> {  
            for(int i=0;i<1000;i++)  
                c.increment();  
        });  
  
        Thread t2 = new Thread(() -> {  
            for(int i=0;i<1000;i++)  
                c.increment();  
        });  
  
        t1.start();  
        t2.start();  
  
        t1.join();  
        t2.join();  
  
        System.out.println(c.count);  
    }  
}
```

Output

```
2000
```

Advantages

- Data consistency

- Thread safety
 - Reliable execution
-

Limitations

- Performance overhead
 - Possible deadlock
-

Q6 (b) Describe common AWT events. (7 Marks)

Definition

AWT events are generated when user interacts with GUI components.

Common Events

Event	Description
ActionEvent	Button click
FocusEvent	Focus gained/lost
KeyEvent	Keyboard action
MouseEvent	Mouse action
WindowEvent	Window action

Event Handling Process

```
User Action
|
Event Generated
|
Event Listener
|
Event Handler Method
```

Example Program

```
import java.awt.*;
import java.awt.event.*;

class Demo extends Frame implements ActionListener {

    Button b;

    Demo() {

        b = new Button("Click Me");

        b.setBounds(100,100,80,30);

        add(b);

        b.addActionListener(this);

        setSize(300,300);
        setLayout(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        System.out.println("Button Clicked");
    }

    public static void main(String args[]) {
        new Demo();
    }
}
```

Output

```
Button Clicked
```

Advantages

- Interactive GUI
 - Event-driven programming
 - Better user experience
-

Q7 (a) Analyze concept of private inheritance in Java and access modifiers. (7 Marks)

Important Point

Java does not support private inheritance directly.

Access Modifiers

Modifier	Same Class	Same Package	Subclass	Other Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Encapsulation Example

```
class Employee {  
  
    private int salary;  
  
    public void setSalary(int salary) {  
        this.salary = salary;  
    }  
  
    public int getSalary() {  
        return salary;  
    }  
}  
  
class Main {  
    public static void main(String args[]) {  
  
        Employee e = new Employee();  
  
        e.setSalary(50000);  
  
        System.out.println(e.getSalary());  
    }  
}
```

```
}  
}
```

Output

```
50000
```

Advantages

- Data hiding
- Better security
- Controlled access

Q7 (b) Write a Java program that reads data from a text file and handles exceptions. (7 Marks)

Definition

File handling is used to read/write files.

Java Program

```
import java.io.*;  
  
class FileRead {  
    public static void main(String args[]) {  
  
        try {  
            FileReader fr = new FileReader("data.txt");  
  
            BufferedReader br = new BufferedReader(fr);  
  
            String line;  
  
            while((line = br.readLine()) != null) {  
                System.out.println(line);  
            }  
        }  
    }  
}
```

```
        br.close();
    }

    catch(FileNotFoundException e) {
        System.out.println("File not found");
    }

    catch(IOException e) {
        System.out.println("Unable to read file");
    }
}
}
```

Expected Output

```
Hello Java
Welcome to File Handling
```

Concepts Used

- FileReader
- BufferedReader
- Exception handling
- try-catch

Advantages

- Robust program
- Handles runtime errors
- Prevents crashing

Q8 (a) Write short notes on Set with example code. (7 Marks)

Definition

Set is a collection that stores unique elements.

Characteristics

- No duplicate elements
 - Unordered collection
 - Part of Collection Framework
-

Types of Set

- HashSet
 - LinkedHashSet
 - TreeSet
-

Example Program

```
import java.util.*;

class Test {
    public static void main(String args[]) {

        HashSet<String> set = new HashSet<String>();

        set.add("Java");
        set.add("Python");
        set.add("Java");

        System.out.println(set);
    }
}
```

Output

```
[Java, Python]
```

Advantages

- Fast searching
 - Duplicate prevention
 - Efficient storage
-

Q8 (b) Write short notes on Vector with example code. (7 Marks)

Definition

Vector is a dynamic array class.

Characteristics

- Dynamic size
 - Synchronized
 - Legacy collection
-

Example Program

```
import java.util.*;

class Test {
    public static void main(String args[]) {

        Vector<String> v = new Vector<String>();

        v.add("Java");
        v.add("Python");
        v.add("C++");

        System.out.println(v);
    }
}
```

Output

```
[Java, Python, C++]
```

Advantages

- Dynamic resizing

- Thread safe
- Easy insertion/deletion

THIRD JAVA EXAM PAPER SOLUTIONS

Q1 (a) Differentiate between Iterator and Enumeration. (2 Marks)

Definition

Iterator and Enumeration are interfaces used for traversing collection elements.

Difference Table

Iterator	Enumeration
Modern interface	Legacy interface
Used with Collection Framework	Used with Vector and Hashtable
Can remove elements	Cannot remove elements
More flexible	Less flexible
Methods: hasNext(), next()	Methods: hasMoreElements(), nextElement()

Example of Iterator

```
import java.util.*;

class Demo {
    public static void main(String args[]) {

        ArrayList<String> list = new ArrayList<String>();

        list.add("Java");
        list.add("Python");

        Iterator<String> itr = list.iterator();

        while(itr.hasNext()) {
```

```
        System.out.println(itr.next());
    }
}
```

Output

```
Java
Python
```

Q1 (b) What is Functional Interface? (2 Marks)

Definition

Functional Interface is an interface containing only one abstract method.

Key Features

- Used in lambda expressions
- Introduced in Java 8
- Can contain default and static methods

Example

```
@FunctionalInterface
interface Demo {
    void show();
}
```

Real Life Applications

- Lambda expressions
- Stream API
- Event handling

Q1 (c) What is JIT Compiler? (2 Marks)

Definition

JIT (Just In Time) compiler converts bytecode into machine code during runtime.

Working

Bytecode → JIT Compiler → Machine Code

Advantages

- Faster execution
 - Improved performance
 - Reduces interpretation overhead
-

Q1 (d) How list in Java is different from normal array? (2 Marks)

Array	List
Fixed size	Dynamic size
Stores primitive values	Stores objects
Faster access	More flexible
Cannot grow dynamically	Can grow dynamically

Example

```
import java.util.*;

class Test {
    public static void main(String args[]) {

        int arr[] = {1,2,3};
```

```
ArrayList<Integer> list = new ArrayList<Integer>();

list.add(1);
list.add(2);
list.add(3);
list.add(4);

System.out.println(arr.length);
System.out.println(list.size());
}
}
```

Output

```
3
4
```

Q1 (e) What is Window Adapter in Swing/AWT? (2 Marks)

Definition

WindowAdapter is an abstract adapter class used to handle window events.

Advantages

- Avoids implementing all WindowListener methods
- Simplifies event handling

Example Program

```
import java.awt.*;
import java.awt.event.*;

class Demo extends Frame {

    Demo() {
```

```
addWindowListener(new WindowAdapter() {  
    public void windowClosing(WindowEvent e) {  
        dispose();  
    }  
});  
  
setSize(300,300);  
setVisible(true);  
}  
  
public static void main(String args[]) {  
    new Demo();  
}  
}
```

Q1 (f) What is Sandbox Model? (2 Marks)

Definition

Sandbox model is Java security mechanism that restricts untrusted code from accessing system resources.

Features

- Controlled execution
- Prevents malicious access
- Safe internet applications

Sandbox Diagram

```
Untrusted Code  
  |  
Sandbox Security  
  |  
Restricted System Access
```

Q1 (g) What is MVC Architecture? (2 Marks)

Definition

MVC stands for:

- Model
- View
- Controller

MVC Diagram

```
graph TD
    User --> View
    View <--> Controller
    Controller <--> Model
```

Advantages

- Better maintenance
- Separation of concerns
- Reusability

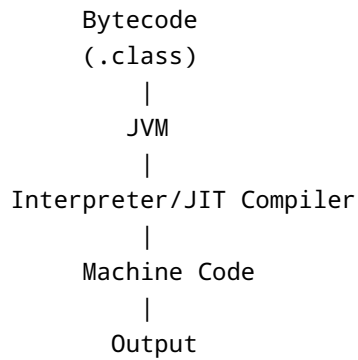
Q2 (a) Analyze Java compilation and execution process. Explain role of JVM, JRE and JDK. (14 Marks)

Definition

Java compilation process converts source code into bytecode which is executed by JVM.

Compilation and Execution Flow Chart

```
graph TD
    A[Java Source File (.java)] --> B[javac]
    B --> C[ ]
```



Stepwise Process

Step 1 – Writing Source Code

```
class Hello {  
    public static void main(String args[]) {  
        System.out.println("Hello Java");  
    }  
}
```

Step 2 – Compilation

Command:

```
javac Hello.java
```

Creates:

```
Hello.class
```

Step 3 – Class Loading

Class Loader loads bytecode.

Step 4 – Bytecode Verification

Checks security and correctness.

Step 5 – Execution

Interpreter/JIT executes code.

Role of JVM

Definition

JVM executes Java bytecode.

Functions

- Memory management
 - Garbage collection
 - Security
 - Bytecode execution
-

Role of JRE

Definition

JRE provides runtime environment.

Components

- JVM
 - Libraries
 - Supporting files
-

Role of JDK

Definition

JDK is complete Java development kit.

Components

- JRE
 - Compiler
 - Debugging tools
-

Relationship Diagram

JDK = JRE + Development Tools
JRE = JVM + Libraries

Advantages

- Platform independent
- Secure execution
- Portable applications

Q2 (b) Analyze structure of Java program and explain role of each component. (14 Marks)

Basic Structure of Java Program

```
// Package Declaration
package mypack;

// Import Statement
import java.util.*;

// Class Declaration
public class Demo {

    // Main Method
    public static void main(String args[]) {

        // Statements
        System.out.println("Hello Java");
    }
}
```

Components Explanation

1. Package Declaration

Used for organizing classes.

Syntax

```
package package_name;
```

2. Import Statement

Imports predefined classes.

Example

```
import java.util.*;
```

3. Class Declaration

Every Java program contains classes.

Syntax

```
class ClassName
```

4. Main Method

Execution starts from main().

Syntax

```
public static void main(String args[])
```

5. Statements

Actual logic of program.

Program Execution Diagram

```
Main Method
```

```
|
```

Program Statements
|
Output

Expected Output

Hello Java

Advantages

- Organized structure
- Easy debugging
- Better readability

Q3 (a) Analyze key features of Java such as platform independence, security and portability. (14 Marks)

1. Platform Independence

Definition

Java programs run on different operating systems without modification.

Working

Java Source → Bytecode → JVM → Output

2. Security

Definition

Java provides secure execution environment.

Security Features

- Bytecode verifier
 - Security manager
 - Sandbox model
 - Garbage collection
-

3. Portability

Definition

Programs can move from one system to another.

Other Important Features

Feature	Description
Object Oriented	Uses classes and objects
Robust	Strong memory management
Multithreading	Concurrent execution
Distributed	Network programming support

Advantages

- Suitable for enterprise applications
 - Cross-platform compatibility
 - Secure internet applications
-

Real Life Applications

- Android development
 - Banking software
 - Web applications
 - Cloud systems
-

Q3 (b) Evaluate role of class loader and bytecode verifier. (14 Marks)

Definition

Class Loader loads class files while Bytecode Verifier checks security.

Class Loader

Functions

- Loads .class files
 - Dynamically loads classes
 - Maintains namespaces
-

Types of Class Loaders

Loader	Work
Bootstrap Loader	Loads core Java classes
Extension Loader	Loads extension libraries
Application Loader	Loads application classes

Class Loader Diagram

```
graph TD
    CF[Class File] --> CL[Class Loader]
    CL --> JM[JVM Memory]
```

Bytecode Verifier

Definition

Checks bytecode for illegal operations.

Checks Performed

- Stack overflow
 - Invalid type conversion
 - Illegal memory access
 - Security violations
-

Bytecode Verification Diagram

```
graph TD; A[Bytecode] --> B[Verifier]; B --> C[Safe Execution];
```

Advantages

- Secure execution
 - Prevents malicious code
 - Maintains JVM integrity
-

Q4 (a) Analyze how inheritance contributes to code reusability in Java. (14 Marks)

Definition

Inheritance allows one class to acquire properties of another class.

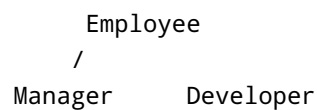
Benefits of Inheritance

- Code reusability
 - Better maintenance
 - Reduces duplication
 - Supports polymorphism
-

Types of Inheritance

Type	Description
Single	One parent one child
Multilevel	Chain inheritance
Hierarchical	One parent many children
Multiple	Through interfaces
Hybrid	Combination of inheritance

Hierarchy Diagram



Java Program

```
class Employee {  
  
    void work() {  
        System.out.println("Employee Working");  
    }  
}  
  
class Manager extends Employee {  
  
    void manage() {  
        System.out.println("Manager Managing Team");  
    }  
}  
  
class Developer extends Employee {  
  
    void code() {  
        System.out.println("Developer Writing Code");  
    }  
}  
  
class Main {  
    public static void main(String args[]) {
```

```
    Manager m = new Manager();
    Developer d = new Developer();

    m.work();
    m.manage();

    d.work();
    d.code();
}
}
```

Output

```
Employee Working
Manager Managing Team
Employee Working
Developer Writing Code
```

Role Based Inheritance

Used in systems where different roles inherit common features.

Examples:

- Student
- Teacher
- Admin

Advantages

- Easy expansion
 - Better software design
 - Reusable code
-

Q4 (b) Design a scenario where method overloading and method overriding are both used. (14 Marks)

Definition

Method overloading uses same method name with different parameters. Method overriding redefines parent method in child class.

Java Program

```
class Animal {  
    void sound() {  
        System.out.println("Animal Sound");  
    }  
}  
  
class Dog extends Animal {  
    // Method overriding  
    void sound() {  
        System.out.println("Dog Barks");  
    }  
    // Method overloading  
    void sound(String type) {  
        System.out.println("Dog Sound Type: " + type);  
    }  
}  
  
class Main {  
    public static void main(String args[]) {  
        Dog d = new Dog();  
        d.sound();  
        d.sound("Loud");  
    }  
}
```

Concepts Used

Method Overriding

Runtime polymorphism.

Method Overloading

Compile-time polymorphism.

Output

```
Dog Barks  
Dog Sound Type: Loud
```

Advantages

- Flexibility
 - Better readability
 - Reusability
-

Q5 (a) Evaluate use of abstract classes and interfaces in real-world applications. (14 Marks)

Abstract Class

Definition

Abstract class contains abstract and non-abstract methods.

Interface

Definition

Interface contains abstract methods and constants.

Difference Table

Abstract Class	Interface
Can have constructors	Cannot have constructors
Partial abstraction	Full abstraction
Uses extends	Uses implements
Can contain instance variables	Only constants

Example Program

```
abstract class Vehicle {  
  
    abstract void start();  
  
    void stop() {  
        System.out.println("Vehicle Stopped");  
    }  
}  
  
interface GPS {  
    void location();  
}  
  
class Car extends Vehicle implements GPS {  
  
    void start() {  
        System.out.println("Car Started");  
    }  
  
    public void location() {  
        System.out.println("GPS Enabled");  
    }  
}  
  
class Main {  
    public static void main(String args[]) {  
  
        Car c = new Car();  
  
        c.start();  
        c.stop();  
        c.location();  
    }  
}
```

```
}  
}
```

Output

```
Car Started  
Vehicle Stopped  
GPS Enabled
```

When to Use Abstract Class?

- Common functionality required
- Shared code among subclasses

When to Use Interface?

- Multiple inheritance needed
- Complete abstraction required

Real Life Applications

- Payment gateways
- Banking systems
- Vehicle systems

Q5 (b) Evaluate role of StringBuffer and StringBuilder in Java. (14 Marks)

Definition

StringBuffer and StringBuilder are mutable string classes.

Comparison Table

Feature	String	StringBuffer	StringBuilder
Mutable	No	Yes	Yes
Thread Safe	Yes	Yes	No
Performance	Slow	Medium	Fast
Synchronization	Automatic	Synchronized	Not synchronized

Example Program

```
class Demo {  
    public static void main(String args[]) {  
  
        StringBuffer sb = new StringBuffer("Java");  
        sb.append(" Programming");  
  
        StringBuilder sb2 = new StringBuilder("Hello");  
        sb2.append(" World");  
  
        System.out.println(sb);  
        System.out.println(sb2);  
    }  
}
```

Output

```
Java Programming  
Hello World
```

StringTokenizer

Definition

Used to split strings into tokens.

Example

```
import java.util.*;

class Demo {
    public static void main(String args[]) {

        StringTokenizer st = new StringTokenizer("Java Python C++");

        while(st.hasMoreTokens()) {
            System.out.println(st.nextToken());
        }
    }
}
```

Output

```
Java
Python
C++
```

Advantages

- Fast string manipulation
- Efficient memory usage
- Better performance

Q6 (a) Analyze how exception handling improves program robustness. (14 Marks)

Definition

Exception handling manages runtime errors.

Exception Handling Flow

try → exception occurs → catch → finally

Keywords

try

Contains risky code.

catch

Handles exceptions.

finally

Always executes.

throw

Used to explicitly throw exception.

throws

Declares exceptions.

Java Program

```
class Demo {  
    public static void main(String args[]) {  
  
        try {  
            int a = 10/0;  
        }  
  
        catch(ArithmeticException e) {  
            System.out.println("Exception Handled");  
        }  
  
        finally {  
            System.out.println("Finally Executed");  
        }  
    }  
}
```

```
}  
}
```

Output

```
Exception Handled  
Finally Executed
```

Advantages

- Prevents abnormal termination
- Better reliability
- Improves robustness

Q6 (b) Evaluate concept of packages in Java. (14 Marks)

Definition

Package is collection of related classes and interfaces.

Roles of Packages

1. Code Organization

Organizes related classes.

2. Reusability

Classes can be reused.

3. Access Control

Controls visibility.

Package Syntax

```
package mypack;
```

Example Program

```
package mypack;

public class Demo {

    public void show() {
        System.out.println("Package Example");
    }
}
```

Advantages

- Avoids naming conflicts
 - Better security
 - Better maintenance
-

Q7 (a) Analyze need for multithreading in modern applications. (14 Marks)

Definition

Multithreading allows multiple threads to execute simultaneously.

Need of Multithreading

- Faster execution
 - Better CPU utilization
 - Responsive applications
 - Parallel processing
-

Thread Life Cycle

New → Runnable → Running → Waiting → Dead

Synchronization Issues

Race Condition

Multiple threads access shared resource.

Deadlock

Threads wait forever.

Java Program

```
class Counter {  
  
    int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
}  
  
class Main {  
    public static void main(String args[]) throws Exception {  
  
        Counter c = new Counter();  
  
        Thread t1 = new Thread(() -> {  
            for(int i=0;i<1000;i++)  
                c.increment();  
        });  
  
        Thread t2 = new Thread(() -> {  
            for(int i=0;i<1000;i++)  
                c.increment();  
        });  
  
        t1.start();  
        t2.start();  
    }  
}
```

```
        t1.join();
        t2.join();

        System.out.println(c.count);
    }
}
```

Output

2000

Real Life Applications

- Gaming
- Banking systems
- Web servers

Q7 (b) Design GUI-based interaction flow using AWT components. (14 Marks)

Definition

AWT is Java GUI toolkit.

Event Handling Flow

```
User Action
  |
Event Generated
  |
Listener
  |
Handler Method
```

Java Program

```
import java.awt.*;
import java.awt.event.*;

class Demo extends Frame implements ActionListener {

    Label l;
    Button b;

    Demo() {

        l = new Label("Click Button");
        l.setBounds(100,50,100,30);

        b = new Button("Submit");
        b.setBounds(100,100,80,30);

        add(l);
        add(b);

        b.addActionListener(this);

        setSize(300,300);
        setLayout(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        l.setText("Button Clicked");
    }

    public static void main(String args[]) {
        new Demo();
    }
}
```

Advantages

- Interactive applications
 - Event-driven programming
 - Better user experience
-

Q8 (a) Analyze Java Collection Framework. (14 Marks)

Definition

Java Collection Framework is set of classes and interfaces for storing data.

Collection Hierarchy

```
Collection
|
List Set Queue
```

Suitable Collection Classes

Scenario	Class
Ordered data	ArrayList
Unique data	HashSet
Key-value data	HashMap
FIFO operations	Queue

Difference: Collection vs Collections

Collection	Collections
Interface	Utility class
Stores data	Provides utility methods

Difference: Collection vs Map

Collection	Map
Stores elements	Stores key-value pairs

Collection	Map
Uses Collection interface	Separate hierarchy

Difference: Comparable vs Comparator

Comparable	Comparator
compareTo()	compare()
Single sorting logic	Multiple sorting logic
Inside class	Separate class

Advantages

- Dynamic storage
- Efficient searching
- Flexible data handling

Q8 (b) Evaluate Reflection in Java. (14 Marks)

Definition

Reflection allows inspection and manipulation of classes at runtime.

Reflection Features

- Access methods
- Access fields
- Dynamic object creation

Example Program

```
import java.lang.reflect.*;

class Student {
    int id = 101;
```

```
void display() {  
    System.out.println("Display Method");  
}  
}  
  
class Demo {  
    public static void main(String args[]) throws Exception {  
  
        Class c = Student.class;  
  
        System.out.println(c.getName());  
  
        Method m[] = c.getDeclaredMethods();  
  
        for(Method x : m) {  
            System.out.println(x.getName());  
        }  
    }  
}
```

Output

```
Student  
display
```

Advantages

- Dynamic programming
- Used in frameworks
- Runtime inspection

Limitations

- Slower execution
- Security risks

Real Life Applications

- Spring Framework
- Hibernate

Q9 (a) Analyze JDBC architecture and CRUD operations. (14 Marks)

JDBC Architecture Diagram

```
graph TD; A[Java Application] --- B[JDBC API]; B --- C[JDBC Driver]; C --- D[Database];
```

Steps of JDBC Connectivity

1. Import package
2. Load driver
3. Establish connection
4. Create statement
5. Execute query
6. Close connection

CRUD Operations

Operation	SQL
Create	INSERT
Read	SELECT
Update	UPDATE
Delete	DELETE

Example Program

```
import java.sql.*;

class Demo {
    public static void main(String args[]) throws Exception {

        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/test","root","root");

        Statement st = con.createStatement();

        st.executeUpdate("insert into student values(101,'Aman')");

        ResultSet rs = st.executeQuery("select * from student");

        while(rs.next()) {
            System.out.println(rs.getInt(1) + " " + rs.getString(2));
        }

        con.close();
    }
}
```

Advantages

- Database connectivity
- Efficient data management
- Platform independent

Q9 (b) Design logic to read text file, convert to uppercase and write into another file. (14 Marks)

Definition

File handling is used for reading and writing files.

File Handling Flow

```
Input File
|
Read Data
|
Convert to Uppercase
|
Write to Output File
```

Java Program

```
import java.io.*;

class Demo {
    public static void main(String args[]) throws Exception {

        FileReader fr = new FileReader("input.txt");
        BufferedReader br = new BufferedReader(fr);

        FileWriter fw = new FileWriter("output.txt");
        BufferedWriter bw = new BufferedWriter(fw);

        String line;

        while((line = br.readLine()) != null) {

            bw.write(line.toUpperCase());
            bw.newLine();
        }

        br.close();
        bw.close();

        System.out.println("File Converted Successfully");
    }
}
```

Output

File Converted Successfully

Classes Used

Class	Purpose
FileReader	Reads file
BufferedReader	Efficient reading
FileWriter	Writes file
BufferedWriter	Efficient writing

Advantages

- Efficient file processing
- Easy data conversion
- Better performance

END OF COMPLETE SOLUTION